ASSIGNMENT 2

Modularization of the Project

Project: AI Research Paper Discovery & Recommendation System

Objective

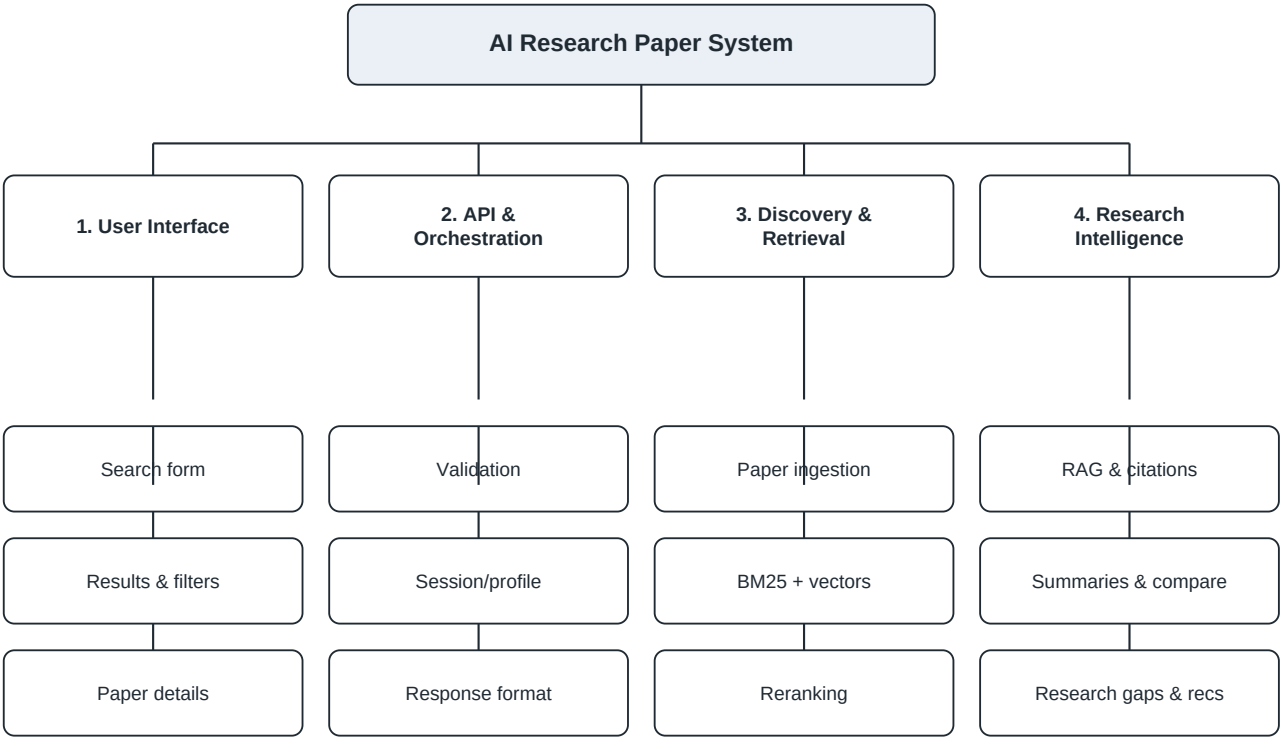
Divide the proposed system into small, well-defined modules with clear responsibilities and interfaces. Modularization should make the system easier to develop, test, maintain, evaluate and extend.

1. System overview

The system accepts a research topic or natural-language question and returns ranked academic papers with explanations. Its intended workflow can include hybrid retrieval, cross-encoder reranking, RAG-based summaries, multi-paper comparison, personalized recommendations, citation-based discovery and research-gap hypotheses. The diagram describes the proposed design; do not claim features as implemented until they are built and tested.

2. Structure chart

The structure chart shows the top-level system, four main functional groups and their submodules. Shared data stores and external services support the modules.



Shared infrastructure: Scholarly APIs • Metadata database • Vector index • LLM/embedding services • Evaluation & logging

High-level modular organization and shared services

3. Functional modules and responsibilities

Module	Purpose and responsibilities	Inputs / outputs
1. User Interface	Accept topic/query, filters and optional preferences. Display ranked papers, metadata, summaries, comparisons, citations and follow-up chat.	Input: query, filters, selected papers. Output: request and rendered results.
2. API & Orchestration	Expose endpoints, validate requests, coordinate stages, manage errors/timeouts and assemble the response.	Input: frontend request. Output: validated workflow result.
3. Paper Ingestion	Fetch scholarly records; normalize fields; deduplicate by DOI/provider ID; store title, abstract, authors, year, references and URL. Process full text only when available and permitted.	Input: API records/PDF. Output: normalized records and extracted text.
4. Query Understanding	Interpret intent, extract concepts, expand terms and build explicit search filters without inventing constraints.	Input: natural-language query. Output: structured query and keywords.
5. Hybrid Retrieval	Run BM25 keyword search and dense semantic search; apply metadata filters; merge and deduplicate candidates.	Input: structured query. Output: candidate papers and retrieval scores.
6. Reranking	Score query-paper pairs with a cross-encoder or suitable reranker, order candidates and select Top-K.	Input: query and candidates. Output: ranked paper list.
7. Evidence Retrieval	Load abstracts or relevant sections/chunks for selected papers and preserve source IDs plus available page/section details.	Input: paper IDs. Output: evidence passages with provenance.
8. RAG Analysis	Generate grounded summaries, methodology descriptions, strengths/limitations and multi-paper comparisons with citations.	Input: question and evidence. Output: cited analytical response.
9. Recommendation & Personalization	Combine relevance with optional interests, reading history and citation signals. Explain why each paper is recommended; use a fallback for new users.	Input: ranked papers and optional profile. Output: recommendations and reasons.
10. Citation / Gap Analysis	Trace citation links when available; identify repeated limitations or under-explored themes. Label proposed directions as hypotheses and support them with evidence.	Input: papers, evidence and citation links. Output: citation relations and gap hypotheses.
11. Data & Storage	Store canonical metadata, optional user preferences/history, extracted text, embeddings and evaluation records with stable identifiers.	Input/output: normalized records, vectors, profiles and evaluation data.
12. Evaluation & Monitoring	Measure retrieval/ranking quality, citation correctness, answer faithfulness, latency, API errors and cost; run baselines and ablations.	Input: test set and system output. Output: metrics and reports.

4. Suggested module interfaces

Interface	Expected contract
understand_query(query, filters)	Returns intent, keywords, expanded terms and explicit metadata filters.
retrieve_candidates(query, filters, top_n)	Returns unique paper IDs, metadata references and keyword/vector scores.
rerank(query, candidates, top_k)	Returns ordered candidates and reranking scores.
get_evidence(paper_ids)	Returns abstracts or relevant chunks with source ID and available section/page details.
generate_grounded_analysis(question, evidence)	Returns response and cited evidence references; omits or qualifies unsupported details.
recommend(papers, user_profile)	Returns ranked papers, recommendation reasons and a no-profile fallback.
evaluate(run_id, test_set)	Returns repeatable retrieval/ranking metrics and RAG quality checks.

5. End-to-end data flow

1. The UI submits a query and optional filters to the API. 2. The API validates the request and calls query understanding. 3. Hybrid retrieval gathers candidates from BM25 and vector search. 4. Reranking orders candidates and selects Top-K. 5. Evidence retrieval obtains abstracts or available full-text sections. 6. RAG generates grounded summaries and comparisons. 7. Recommendation uses query relevance and optional user preferences or citation signals. 8. The API returns papers with source links and explanations. 9. Evaluation tools run offline against labelled queries and report quality metrics.

6. Error handling and module boundaries

• If a scholarly API is unavailable, use cached records where appropriate and identify missing or potentially stale metadata.

• If full text is unavailable, continue with the abstract and label the output as abstract-based.

• If vector search fails, a keyword-search fallback can preserve basic discovery.

• If an LLM call fails or returns malformed output, validate the response and retry within a limit; otherwise return a controlled error or search-only results.

• Keep provider-specific code behind adapters so the rest of the application does not depend on one API’s response format.

7. Testing plan

Unit tests should cover query parsing, metadata normalization, deduplication, score fusion, reranker output, citation mapping and response formatting. Integration tests should exercise the complete query-to-results workflow on a small fixed corpus. Compare BM25, dense retrieval, hybrid retrieval and hybrid plus reranking using Recall@K, Precision@K, MRR and nDCG@K. Check RAG citation correctness and faithfulness to retrieved evidence. Record latency and external API failures.

8. Conclusion

This modular design separates the UI, orchestration, ingestion, query understanding, retrieval, ranking, evidence retrieval, RAG, recommendation and evaluation. Clear contracts allow modules to be improved or replaced without rewriting the entire application. For the first version, these modules can live inside one FastAPI application as a modular monolith.

References / further reading

1. Sommerville, I. (2015). *Software Engineering* (10th ed.). Pearson.

2. Martin, R. C. (2017). *Clean Architecture: A Craftsman’s Guide to Software Structure and Design.* Prentice Hall.

3. Bass, L., Clements, P., & Kazman, R. (2021). *Software Architecture in Practice* (4th ed.). Addison-Wesley.